DEVSECOPS PIPELINE FOR SECURE SOFTWARE DELIVERY PLATFORM

SOFTWARE ARCHITECTURE DESIGN REPORT

CO2 ASSESSMENT TOOL 2

M.Tharani(192472375)

1. INTRODUCTION

1.1 Project Overview

The DevSecOps Pipeline for Secure Software Delivery Platform is a software architecture designed to integrate security practices directly into the software development lifecycle. The platform automates the process of building, testing, scanning, and deploying software while embedding security checks at every stage, rather than treating security as a separate, late-stage activity. This report presents the architectural design of the platform, covering requirement analysis, architectural style selection, component design, quality attribute analysis, and key architectural decisions.

1.2 Purpose of the Architecture

The purpose of this architecture is to provide a scalable, secure, and maintainable foundation for continuous integration and continuous delivery (CI/CD) that incorporates automated security scanning as a first-class concern. The architecture aims to reduce the time between code commit and secure production deployment while minimizing the risk of shipping vulnerable code, exposed secrets, or insecure container images.

1.3 System Objectives

- Automate the build, test, and deployment process for software projects.
- Integrate static code analysis, dependency scanning, secret scanning, and container scanning into the pipeline.
- Provide real-time visibility into pipeline execution and security findings.
- Ensure the platform can scale to support growing numbers of projects, users, and pipeline executions.
- Maintain a complete and auditable record of pipeline and security events.

1.4 Scope of the Proposed System

The scope of the proposed system covers the end-to-end DevSecOps pipeline, from source code integration through to deployment and monitoring. It includes user and project management, integration with source control platforms such as GitHub, automated build and test execution, multiple layers of security scanning, artifact storage, container orchestration through Kubernetes, and monitoring and notification capabilities. The scope excludes the

internal implementation details of third-party tools such as SonarQube and Trivy, which are treated as integrated services within the architecture.

2. SYSTEM REQUIREMENT ANALYSIS

2.1 Functional Requirements

The following table summarizes the core functional requirements of the platform.

ID	Functional Requirement
FR-01	The system shall allow developers to authenticate and manage projects through a web dashboard.
FR-02	The system shall integrate with GitHub to detect code pushes and pull requests automatically.
FR-03	The system shall trigger an automated CI/CD pipeline upon detecting a new commit.
FR-04	The system shall execute build and unit/integration testing stages automatically.
FR-05	The system shall perform static code analysis using SonarQube on every build.
FR-06	The system shall scan third-party dependencies for known vulnerabilities.
FR-07	The system shall scan repositories and configuration files for exposed secrets.
FR-08	The system shall scan container images for vulnerabilities before deployment using Trivy.
FR-09	The system shall store verified build artifacts in a centralized artifact repository.
FR-10	The system shall deploy approved artifacts to a Kubernetes cluster automatically.
FR-11	The system shall continuously monitor deployed services and infrastructure health.

ID	Functional Requirement
FR-12	The system shall send real-time notifications on pipeline status and security findings.
FR-13	The system shall maintain an immutable audit log of all pipeline and security events.

2.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality constraints the system must satisfy in addition to its functional behavior.

Attribute	Description
Scalability	The platform must scale horizontally to support increasing numbers of concurrent pipelines and microservices.
Security	All services must enforce authentication, authorization, encryption in transit, and secret management.
Reliability	The platform must recover gracefully from service failures without pipeline data loss.
Availability	Core services must maintain high availability, targeting 99.9 percent uptime.
Performance	Pipeline execution and security scans must complete within acceptable time thresholds to support fast delivery.
Maintainability	Individual services must be independently deployable, testable, and upgradable with minimal impact on others.

2.3 Key Quality Attributes

- **Scalability:** The system must support horizontal growth in users, projects, and pipeline load without architectural redesign.

- **Security:** The system must enforce authentication, authorization, encrypted communication, and continuous vulnerability scanning.
- **Reliability:** The system must continue to function correctly in the presence of partial failures.
- **Availability:** Core services must remain accessible with minimal downtime.
- **Performance:** Pipeline stages must execute within predictable and acceptable time limits.
- **Maintainability:** Components must be independently modifiable, testable, and deployable.

3. SELECTION OF SOFTWARE ARCHITECTURE

The proposed system adopts a Hybrid Architecture that combines Microservices Architecture with Event-Driven Architecture. This hybrid style was selected because the DevSecOps platform requires both independently scalable, well-bounded services (such as authentication, build, and security scanning) and asynchronous, loosely coupled communication for pipeline events (such as build completion, scan results, and deployment status).

3.1 Why This Architecture Is Suitable

- Microservices allow each capability, such as build execution or container scanning, to be developed, deployed, and scaled independently.
- Event-driven communication through a message broker decouples pipeline stages, allowing services to react to events such as "build completed" or "vulnerability detected" without direct dependencies.
- The combination supports the continuous, automated, and security-integrated nature of a DevSecOps pipeline, where many stages must run in sequence or in parallel and communicate asynchronously.

3.2 Comparison with Alternative Architectures

Architecture	Strengths	Limitations for This Project
Layered Architecture	Simple to understand	Tight coupling between layers; difficult to scale individual

Architecture	Strengths	Limitations for This Project
		capabilities; slow deployment cycles
MVC	Good separation of presentation logic	Designed for single applications, not distributed backend services; lacks native support for asynchronous events
Client-Server	Straightforward request-response model	Central server becomes a bottleneck; limited scalability and fault isolation
Monolithic Architecture	Simple initial deployment	Poor scalability, slow build/test cycles, single point of failure, difficult to adopt DevSecOps practices
Hybrid (Microservices + Event-Driven)	Independent scalability, fault isolation, asynchronous processing, supports continuous delivery and security automation	Higher initial design and operational complexity, requiring service orchestration and monitoring

As shown in the comparison table, the Hybrid Architecture of Microservices and Event-Driven communication provides the strongest combination of scalability, fault isolation, and support for the asynchronous, security-embedded workflow required by a DevSecOps platform, making it the most suitable choice for this project.

4. SOFTWARE ARCHITECTURE DIAGRAM

The following diagrams represent the complete software architecture of the DevSecOps Pipeline platform, including all major components and their interactions. The Mermaid diagram can be rendered directly in Markdown-compatible editors, and the PlantUML diagram can be rendered using any PlantUML-compatible tool.

4.1 Mermaid Diagram

```
flowchart TD
    U[User] --> WD[Web Dashboard]
    WD --> GW[API Gateway]
    GW --> AUTH[Authentication Service]
    GW --> UM[User Management Service]
    GW --> PM[Project Management Service]
    PM --> GH[GitHub Integration]
    GH --> CICD[CI/CD Pipeline Service]
    CICD --> BUILD[Build Service]
    BUILD --> TEST[Testing Service]
    TEST --> SCA[Static Code Analysis - SonarQube]
    TEST --> DEP[Dependency Scanner]
    TEST --> SEC[Secret Scanner]
    BUILD --> ART[Artifact Repository]
    ART --> CSCAN[Container Security Scanner - Trivy]
    CSCAN --> DEPLOY[Deployment Service]
    DEPLOY --> K8S[Kubernetes Cluster]
    K8S --> MON[Monitoring Service]
    CICD --> BROKER[(Message Broker / Event Bus)]
    BROKER --> MON
    BROKER --> NOTIFY[Notification Service]
    BROKER --> AUDIT[Audit Log Service]
    MON --> NOTIFY
    NOTIFY --> U
    AUDIT --> DB[(Database)]
    UM --> DB
    PM --> DB
    CICD --> DB
```

4.2 PlantUML Diagram

```
@startuml
actor User
rectangle "Web Dashboard" as WD
rectangle "API Gateway" as GW
rectangle "Authentication Service" as AUTH
rectangle "User Management Service" as UM
rectangle "Project Management Service" as PM
rectangle "GitHub Integration" as GH
rectangle "CI/CD Pipeline Service" as CICD
rectangle "Build Service" as BUILD
rectangle "Testing Service" as TEST
rectangle "Static Code Analysis (SonarQube)" as SCA
rectangle "Dependency Scanner" as DEP
rectangle "Secret Scanner" as SEC
rectangle "Container Security Scanner (Trivy)" as CSCAN
rectangle "Artifact Repository" as ART
rectangle "Deployment Service" as DEPLOY
rectangle "Kubernetes Cluster" as K8S
rectangle "Monitoring Service" as MON
queue "Message Broker / Event Bus" as BROKER
rectangle "Notification Service" as NOTIFY
rectangle "Audit Log Service" as AUDIT
database "Database" as DB

User --> WD
WD --> GW
GW --> AUTH
GW --> UM
```

```
GW --> PM
PM --> GH
GH --> CICD
CICD --> BUILD
BUILD --> TEST
TEST --> SCA
TEST --> DEP
TEST --> SEC
BUILD --> ART
ART --> CSCAN
CSCAN --> DEPLOY
DEPLOY --> K8S
K8S --> MON
CICD --> BROKER
BROKER --> MON
BROKER --> NOTIFY
BROKER --> AUDIT
MON --> NOTIFY
NOTIFY --> User
AUDIT --> DB
UM --> DB
PM --> DB
CICD --> DB
@enduml
```

5. COMPONENT DESCRIPTION AND INTERACTION

The table below describes the purpose, responsibilities, inputs and outputs, and interactions of each major component in the architecture.

Component	Purpose	Responsibilities	Inputs / Outputs	Interacts With
User	Initiate development and deployment activities	Write and push code; review pipeline results	Source code, commands / Pipeline status, reports	Web Dashboard
Web Dashboard	Central interface for developers and administrators	Display pipeline status, project settings, security reports	User actions, API responses / UI views, requests	API Gateway
API Gateway	Single entry point for all client requests	Route requests, enforce rate limiting, aggregate responses	Client HTTP requests / Routed requests to services	Authentication Service, all backend services
Authentication Service	Verify identity of users and services	Issue and validate tokens, manage sessions	Credentials, tokens / Auth tokens, validation results	API Gateway, User Management Service
User Management Service	Manage user accounts and roles	Create, update, and authorize user profiles	User data, role requests / User records, permissions	Authentication Service, Project Management Service
Project Management Service	Manage project metadata and configuration	Store project settings, pipeline configuration	Project data / Project records	GitHub Integration, CI/CD Pipeline Service

Component	Purpose	Responsibilities	Inputs / Outputs	Interacts With
GitHub Integration	Connect the platform to source repositories	Detect commits, pull requests, and webhooks	Repository events / Commit metadata, triggers	Project Management Service, CI/CD Pipeline Service
CI/CD Pipeline Service	Orchestrate the end-to-end delivery pipeline	Coordinate build, test, scan, and deploy stages	Trigger events, pipeline definitions / Stage execution commands	Build Service, Testing Service, Security Scanners, Message Broker
Build Service	Compile and package application code	Execute build scripts, generate build artifacts	Source code / Build artifacts	CI/CD Pipeline Service, Artifact Repository
Testing Service	Validate functional correctness of the application	Run unit, integration, and regression tests	Build artifacts, test suites / Test reports	CI/CD Pipeline Service, Notification Service
Static Code Analysis (SonarQube)	Identify code quality and security issues in source code	Analyze code for bugs, code smells, vulnerabilities	Source code / Quality and security reports	CI/CD Pipeline Service, Audit Log Service
Dependency Scanner	Detect vulnerabilities in third-party libraries	Scan dependency manifests against vulnerability databases	Dependency manifests / Vulnerability report	CI/CD Pipeline Service, Notification Service

Component	Purpose	Responsibilities	Inputs / Outputs	Interacts With
Secret Scanner	Prevent leakage of credentials and keys	Scan source code and configuration for exposed secrets	Source code, config files / Secret findings report	CI/CD Pipeline Service, Audit Log Service
Container Security Scanner (Trivy)	Assess container image security before deployment	Scan container images for vulnerabilities and misconfigurations	Container images / Scan report	CI/CD Pipeline Service, Deployment Service
Artifact Repository	Store verified build outputs and container images	Version, store, and serve build artifacts	Build artifacts / Stored artifact references	Build Service, Deployment Service
Deployment Service	Deploy verified artifacts to the runtime environment	Apply deployment manifests, manage rollout strategy	Artifact reference, deployment config / Deployment status	Artifact Repository, Kubernetes Cluster
Kubernetes Cluster	Host and orchestrate running application containers	Schedule, scale, and manage containerized workloads	Deployment manifests / Running services	Deployment Service, Monitoring Service
Monitoring Service	Observe system health and performance	Collect metrics, logs, and traces from running services	Runtime metrics, logs / Alerts, dashboards	Kubernetes Cluster, Notification Service

Component	Purpose	Responsibilities	Inputs / Outputs	Interacts With
Notification Service	Inform stakeholders of pipeline and security events	Send alerts via email, chat, or webhook	Event messages / Notifications	Message Broker, Users
Audit Log Service	Maintain a tamper-evident record of platform activity	Persist events related to security and pipeline actions	Event data from all services / Audit records	Database, Message Broker
Database	Persist structured platform data	Store user, project, pipeline, and audit records	Write requests / Query results	Most backend services
Message Broker / Event Bus	Enable asynchronous, decoupled communication between services	Publish and route events between producers and consumers	Published events / Delivered events	CI/CD Pipeline Service, Monitoring Service, Notification Service, Audit Log Service

5.1 End-to-End Workflow

The following sequence describes the complete workflow of the DevSecOps pipeline, from code push to deployment and monitoring.

1. A developer pushes code to the GitHub repository.
2. GitHub Integration detects the push event and notifies the CI/CD Pipeline Service through the Project Management Service.
3. The CI/CD Pipeline Service starts a new pipeline run and invokes the Build Service.
4. The Build Service compiles the source code and produces build artifacts.

5. The Testing Service executes unit, integration, and regression tests against the build artifacts.
6. Security scans are executed in parallel: Static Code Analysis (SonarQube) inspects source code quality and security issues, the Dependency Scanner checks for vulnerable libraries, and the Secret Scanner checks for exposed credentials.
7. If all tests and scans pass, the verified artifact is stored in the Artifact Repository.
8. The Container Security Scanner (Trivy) scans the resulting container image before deployment is authorized.
9. The Deployment Service deploys the approved artifact to the Kubernetes Cluster.
10. The Monitoring Service continuously observes the health and performance of the deployed services.
11. The Notification Service sends status updates and alerts to the developer through the Message Broker / Event Bus.
12. The Audit Log Service records every pipeline and security event, persisting them in the Database for compliance and traceability.

6. QUALITY ATTRIBUTE ANALYSIS

The table below explains how the proposed architecture satisfies each key quality attribute and the corresponding expected benefit.

Quality Attribute	**Architectural Support**	**Expected Benefit**
Scalability	Independent microservices scale horizontally based on load; event-driven communication decouples producers from consumers.	The platform handles increasing numbers of concurrent pipelines and users without redesigning the whole system.
Security	Dedicated security scanning services (SAST, dependency, secret, container) are embedded directly into the pipeline; API Gateway	Vulnerabilities are detected early, reducing the risk of insecure code reaching production.

Quality Attribute	Architectural Support	Expected Benefit
	centralizes authentication and authorization.	
Performance	Asynchronous event processing avoids blocking calls; services can be scaled independently to relieve bottlenecks.	Pipelines execute efficiently even as scanning and build workloads grow.
Reliability	The message broker provides durable, retryable event delivery; failures in one service do not directly crash others.	The platform continues operating correctly despite partial failures.
Availability	Stateless, independently deployable services can be replicated across nodes in the Kubernetes cluster.	Reduced downtime and continuous availability of core delivery functions.
Maintainability	Clear service boundaries and single-responsibility components allow isolated updates and testing.	Teams can modify or replace individual services with minimal impact on the rest of the platform.

7. ARCHITECTURAL DECISIONS AND JUSTIFICATION

7.1 Why Hybrid Architecture Was Selected

The Hybrid Architecture was selected because it directly addresses the two dominant forces in this system: the need for independently scalable, security-focused services, and the need for asynchronous, decoupled communication across a multi-stage pipeline. Pure microservices alone would still require synchronous coordination for pipeline events, while a purely event-driven design without service boundaries would make it difficult to scale or secure individual capabilities such as authentication or scanning.

7.2 Important Architectural Decisions

- Use of an API Gateway as the single entry point to simplify client interaction and centralize authentication enforcement.
- Use of a Message Broker / Event Bus to decouple the CI/CD Pipeline Service from downstream consumers such as Monitoring, Notification, and Audit Log services.
- Separation of security scanning into distinct services (static analysis, dependency scanning, secret scanning, container scanning) to allow independent evolution and scaling of each scanning capability.
- Use of Kubernetes as the deployment target to provide container orchestration, scaling, and self-healing capabilities.

7.3 Trade-offs Considered

- Increased operational complexity from managing many independent services versus the flexibility and fault isolation gained.
- Eventual consistency introduced by asynchronous event processing versus the improved throughput and decoupling it provides.
- Additional infrastructure required for service orchestration and monitoring versus the long-term maintainability benefits.

7.4 Future Scalability

The microservices can be scaled independently based on load, for example by increasing replicas of the Testing Service or the Container Security Scanner during periods of high pipeline activity, without needing to scale the entire platform.

7.5 Integration with Cloud Platforms

Because the architecture is container-based and orchestrated through Kubernetes, it can be deployed on any major cloud provider offering managed Kubernetes services, allowing the platform to take advantage of cloud-native scaling, storage, and identity management capabilities.

7.6 Long-Term Maintainability

The clear separation of responsibilities across services, combined with well-defined event contracts on the Message Broker, allows individual components to be updated, replaced, or

extended with minimal impact on the rest of the system, supporting long-term maintainability.

7.7 Security Advantages of the DevSecOps Architecture

- Security scanning is embedded directly into the pipeline rather than performed as a separate, late-stage process.
- Multiple independent scanning layers (code, dependencies, secrets, containers) provide defense in depth.
- Centralized authentication through the API Gateway and Authentication Service reduces the attack surface across services.
- The Audit Log Service provides traceability for compliance and incident investigation.

8. CONCLUSION

This report has presented the software architecture design for the DevSecOps Pipeline for Secure Software Delivery Platform. The Hybrid Architecture, combining Microservices and Event-Driven communication, was shown to be the most suitable choice for this system due to its support for independent scalability, fault isolation, and the asynchronous, security-integrated workflow required by modern DevSecOps practices. The architecture satisfies the key quality attributes of scalability, security, reliability, availability, performance, and maintainability through clearly defined service boundaries, embedded security scanning, and decoupled event-driven communication.

Future enhancements to the platform could include the addition of automated policy-as-code enforcement, integration of runtime security monitoring, support for multi-cloud deployment strategies, and the introduction of machine-learning-based anomaly detection within the Monitoring Service to further strengthen the platform's security posture.